Migración de conexiones TCP en *proxies* balanceadores de carga

Capítulo 1 · Proyecto Integrador

Braulio de Jesús Rivas Abad
Ingeniería en Ciencias Computacionales · ESPOL

Contexto del problema

- Un *proxy* recibe conexiones de clientes y las enruta hacia servidores de destino.
- En la capa 4 administra direcciones y puertos; en la capa 7 interpreta protocolos como HTTP.
- Durante una actualización puede ser necesario reemplazar el proceso mientras existen conexiones activas.
- Cloudflare y GitHub documentan las dificultades de reiniciar servicios con conexiones de larga duración.

El problema es retirar el proceso anterior sin interrumpir las comunicaciones que ya están en curso.

Urgencia de aplicar parches

Mandiant analizó **41 vulnerabilidades** divulgadas en 2023 cuya primera explotación ocurrió después de estar disponible un parche.

Tiempo desde la disponibilidad del parche	Porcentaje acumulado explotado
Hasta una semana	**29 %**
Hasta un mes	**56 %**

Estos datos respaldan la importancia de desplegar correcciones oportunamente, aunque **no miden el riesgo específico de los *proxies***.

Fuente: Mandiant, análisis publicado en 2024.

Drenaje, migración y justificación

Estrategia	Qué ocurre con las conexiones activas	Consecuencia
Drenaje	El proceso anterior espera a que terminen	Puede retrasar la actualización
Migración	El estado se transfiere a un proceso nuevo	El anterior puede retirarse antes

La migración es relevante cuando no resulta aceptable esperar el cierre natural de las conexiones.

Objetivo general

Desarrollar una biblioteca en Rust que migre conexiones TCP activas entre procesos de un *proxy* en la misma máquina, preservando el estado TLS/HTTP y manteniendo la continuidad de la comunicación.

Objetivos específicos

1. Identificar el estado de los protocolos de comunicación que debe preservarse tal que garantice la continuidad de las conexiones durante su migración entre procesos.
2. Diseñar la arquitectura de la biblioteca y el formato de intercambio de estado entre las instancias de origen y destino del *proxy*.
3. Implementar la transferencia del *socket* TCP y los adaptadores necesarios para la migración del estado TLS y HTTP en un *proxy* de demostración.
4. Evaluar el prototipo mediante pruebas de continuidad de conexiones, tiempo de migración y sobrecarga durante una actualización del *proxy*.

Alcance de la propuesta

- Procesos cooperantes en la **misma máquina Linux** y con formatos de estado compatibles.
- Conservación de la conexión cliente–*proxy* y de la conexión *proxy*–servidor.
- Sesiones TLS 1.3 ya establecidas y HTTP/1.1.
- Solicitudes y respuestas con longitud conocida o codificación por fragmentos; una transacción por par de conexiones.

No se incluyen migración entre máquinas, fallos abruptos, *handshake* TLS, *pipelining* HTTP ni actualizaciones de protocolo.

Puntos de integración y puntos seguros

- **Punto de integración:** lugar del *proxy* donde la biblioteca recibe o entrega el estado de una conexión.
- **Punto seguro:** instante entre operaciones de E/S en el que ese estado puede capturarse sin perder bytes ni alterar el orden.

Ejemplo: durante una descarga incompleta, el *proxy* se detiene en un punto seguro, transfiere el estado y el proceso nuevo continúa la misma respuesta. No espera a que termine todo el cuerpo HTTP.

Estado que debe conservarse

Capa	Estado relevante
TCP	*Sockets* de ambos lados de la comunicación
TLS	Secretos, contadores, registros pendientes y estado de cifrado
HTTP	Progreso del mensaje y bytes pendientes
Proxy	Asociación entre la conexión del cliente y la del servidor

La continuidad depende de recuperar el estado de forma coherente en todas estas capas.

Antecedentes y adaptación pendiente

Trabajo	Aporte principal	Aspecto pendiente
Ecdysis	Manejo de conexiones aceptadas y pendientes	Integración con el flujo del *proxy*
CRIU/COAT	Captura y restauración de estado	Compatibilidad e interpretación del estado
MOSN	Conservación de conexiones cliente y *upstream*	Migración del estado interno de protocolos
Capybara	Migración de TCP y TLS	HTTP, Linux e integración de ambas conexiones

Los antecedentes muestran la viabilidad de componentes aislados; la propuesta busca integrarlos para el escenario delimitado.

Aporte propuesto

- Protocolo de coordinación entre el proceso de origen y el de destino.
- Adaptadores para transferir y restaurar el estado de TLS y HTTP.
- Interfaz para que el *proxy* indique puntos seguros de migración.
- *Proxy* de demostración y pruebas de continuidad.

Las bibliotecas y mecanismos concretos se seleccionarán y justificarán en el capítulo 2.

Evaluación prevista

Criterio	Evidencia esperada
Continuidad	La conexión sigue sin abrir una conexión nueva ni repetir TLS
Integridad y orden	No hay pérdida ni duplicación de datos
Retiro del origen	El proceso anterior termina tras la confirmación del destino
Costo	Tiempo de migración y uso de CPU/memoria

Se probarán cargas y descargas con cuerpos incompletos, escrituras parciales y búferes pendientes, y se comparará migración, drenaje y ausencia de actualización.

Referencias principales

- Cloudflare. *Oxy: the journey of graceful restarts.*
- GitHub Engineering. *HAProxy zero-downtime, zero-delay reloads with multibinder.*
- Mandiant. *Time to exploit trends: 2023 year in review.*
- MOSN. *Smooth upgrade.*
- Capybara. *Migrating live TCP/TLS connections.*
- RFC 9293, *Transmission Control Protocol*.
- RFC 8446, *The Transport Layer Security (TLS) Protocol Version 1.3*.
- RFC 9112, *HTTP/1.1*.